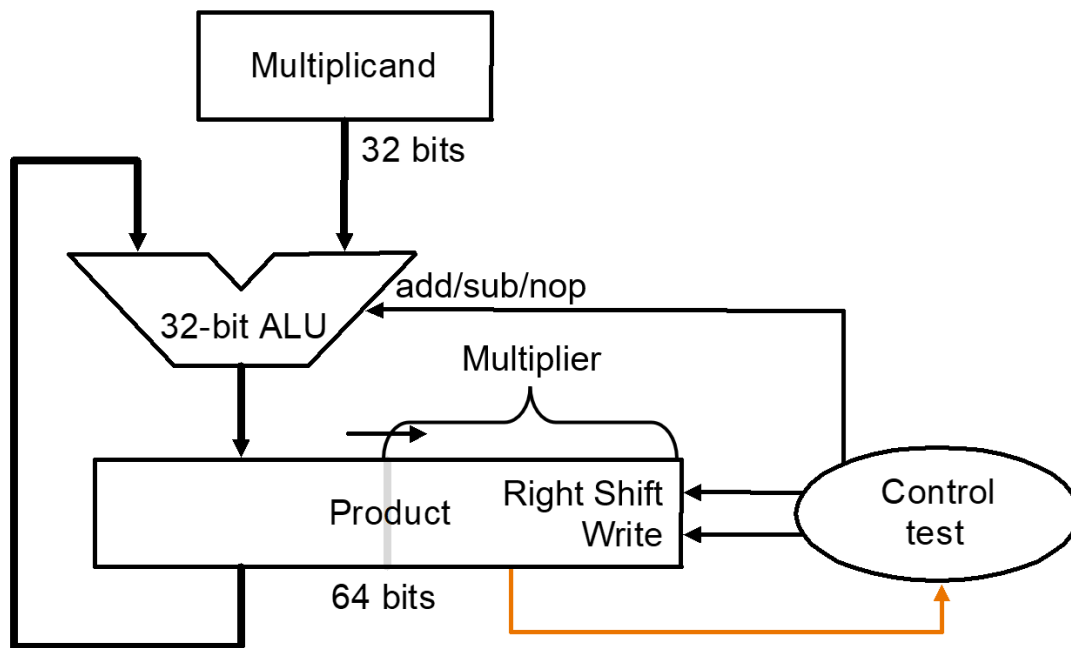


Department of Electrical and Computer Engineering
University of Wisconsin – Madison
ECE 552 Introduction to Computer Architecture
In-Class Exercise (12/03)

1. You modify the improved 32-bit multicycle multiplier for **signed integers** described in class, implementing **Booth's multiplication** algorithm instead. The hardware organization is shown below. The multiplier operand is initialized into the least-significant half of the product register; the most-significant half is initialized to 0. For each bit of the multiplier operand, the control logic computes Booth's encoding then performs two tasks: 1) either does nothing (nop) or adds/subtracts the multiplicand onto the most-significant half of the product register based on Booth's encoding; then 2) right shifts the 64-bit product register by one. Unfortunately, you have forgotten if the right shift should be **logical** or **arithmetic**.



- a) Using the notation from class, provide the **Booth's encoding** for this 32-bit number: -8.

Answer:
00000000 00000000 00000000 0000-000

- b) Using the notation from class, provide the **Booth's encoding** for this 32-bit number: 5.

Answer:
00000000 00000000 00000000 0000+-+--

- c) Fill in the two tables below for two example **32-bit multiplications** running on this hardware: -8×-2 and 5×4 . Each row corresponds to a bit position in the multiplier being processed. At each row, specify what **operation** (add/sub/nop) is performed and what the contents of the 64-bit **product register** are after performing the right shift (either logical or arithmetic). The first couple rows are provided for you.

multiplier = $-8 \times \text{multiplicand} = -2$			
		product register (in hex)	
bit	add/sub/nop?	after logical shift	after arithmetic shift
init		00000000 FFFFFFFF8	00000000 FFFFFFFF8
0	nop	00000000 7FFFFFFC	00000000 7FFFFFFC
1	nop	00000000 3FFFFFFE	00000000 3FFFFFFE
2	nop	00000000 1FFFFFFF	00000000 1FFFFFFF
3	sub	00000001 0FFFFFFF	00000001 0FFFFFFF
...			
31	nop	00000000 00000010	00000000 00000010

multiplier = $5 \times \text{multiplicand} = 4$			
		product register (in hex)	
bit	add/sub/nop?	after logical shift	after arithmetic shift
init		00000000 00000005	00000000 00000005
0	sub	7FFFFFFE 00000002	FFFFFFFE 00000002
1	add	40000001 00000001	00000001 00000001
2	sub	1FFFFFFE 80000000	FFFFFFFE 80000000
3	add	10000001 40000000	00000001 40000000
...			
31	nop	00000001 00000014	00000000 00000014

2. The following MIPS assembly program performs **unsigned integer division** of two 16-bit numbers: $19 \div 3$ (in decimal).

```
L0:    lui    $t0, 3
L1:    ori    $t1, $zero, 19
L2:    add    $t2, $zero, $zero
L3:    addi   $t3, $zero, 16
L4:    srl    $t0, $t0, 1
L5:    sll    $t2, $t2, 1
L6:    sltu   $t7, $t1, $t0
L7:    bne    $t7, $zero, L10
L8:    subu   $t1, $t1, $t0
L9:    ori    $t2, $t2, 1
L10:   addi   $t3, $t3, -1
L11:   bne    $t3, $zero, L4
```

- a) Which register contains the **quotient** at the end of this program?

Answer:

Register \$t2

- b) Which register contains the **remainder** at the end of this program?

Answer:

Register \$t1

- c) How many times is **instruction L8** executed in this program?

Answer:

2 times